

비동기 JavaScript와 API 연동

실행 환경
- 브라우저
- Node.js

브라우저의 기능

학습 목표: - 동기·비동기의 차이와 **이벤트 루프**의 동작 원리를 설명할 수 있습니다 - **콜백**과 **프로미스**로 비동기 작업을 다룰 수 있습니다 - **async/await**와 **try/catch**로 가독성·안정성 더 중요
좋은 비동기 코드를 작성할 수 있습니다 - Fetch API로 외부 데이터를 가져와 화면에 렌더링하는 앱을 만들 수 있습니다

javascript는 단일스레드 방식

동기와 비동기, 이벤트 루프

도입

지금까지 작성한 코드는 모두 위에서 아래로 순서대로 실행되었습니다. 그런데 웹 페이지는 서버에서 데이터를 받아오거나, 사용자의 클릭을 기다리거나, 타이머가 끝날 때까지 대기하는 상황이 자주 생깁니다. 이런 "기다림"을 동기(Synchronous) 방식으로 처리하면 브라우저 전체가 얼어붙어 버립니다. 이를 해결하는 것이 비동기(Asynchronous) 프로그래밍입니다.

핵심 개념 설명

동기(Synchronous)와 비동기(Asynchronous)

동기(Synchronous) 방식은 **앞 작업이 완전히 끝나야 다음 작업을 시작합니다**. 은행 창구에서 한 명씩 업무를 처리하는 모습과 같습니다.

비동기(Asynchronous) 방식은 시간이 **오래 걸리는 작업을 "맡겨 두고" 다른 일을 먼저 처리**합니다. **세탁기를 돌려 놓고 밥을 짓는 것과** 같습니다. **세탁이 끝나면 알림이 오고, 그때 빨래를 꺼내면** 됩니다.

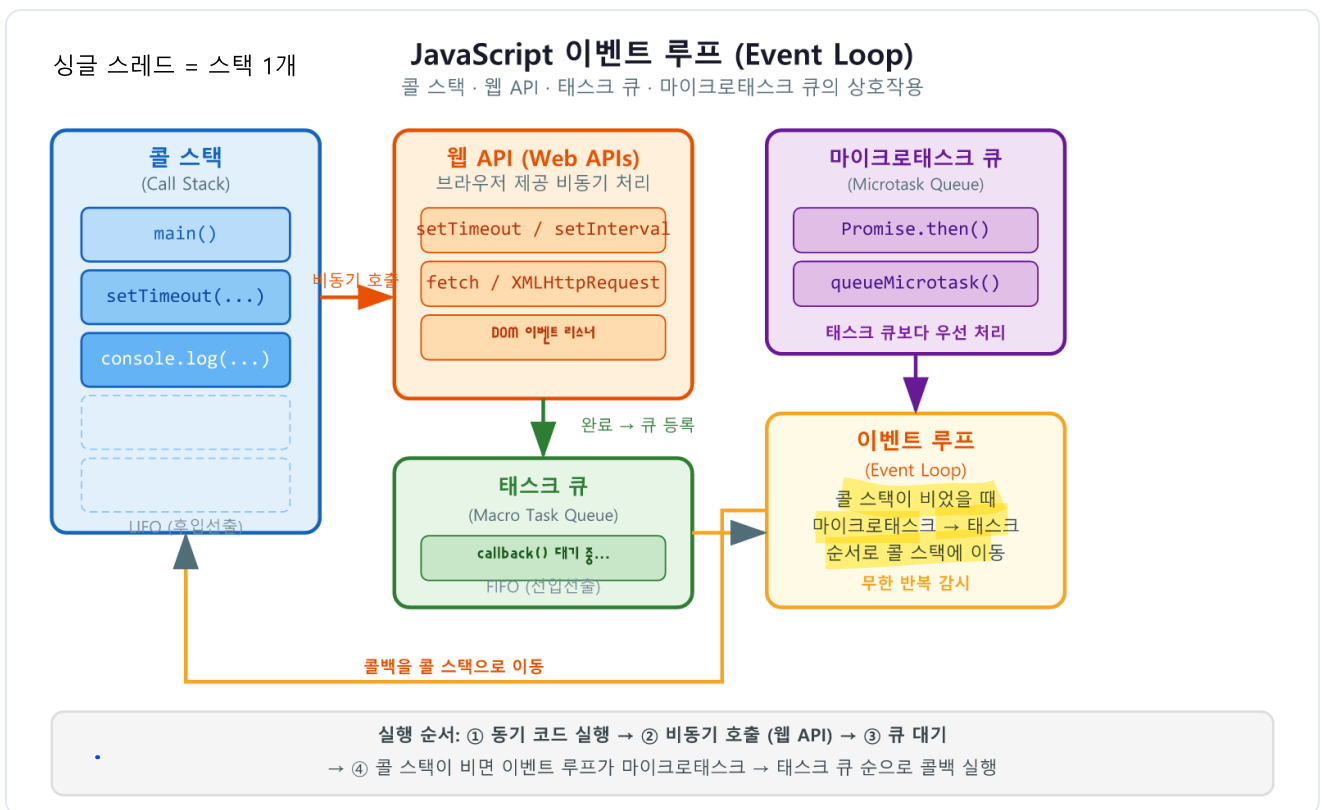
이벤트 루프(Event Loop)가 작업을 처리하는 순서

JavaScript는 **단일 스레드(Single Thread)** 언어입니다. 한 번에 하나의 작업만 처리할 수 있습니다. 그런데도 여러 일을 동시에 하는 것처럼 느껴지는 이유가 바로 **이벤트 루프(Event Loop)** 덕분입니다.

이벤트 루프는 다음 세 가지 구성 요소로 작동합니다.

- **콜 스택(Call Stack):** 현재 실행 중인 함수들이 쌓이는 곳입니다. 함수가 호출되면 쌓이고, 실행이 끝나면 사라집니다. 스택 구조라
- **웹 API(Web API):** 브라우저가 제공하는 비동기 기능입니다. `setTimeout`, `fetch`, 이벤트 리스너 등이 여기에서 처리됩니다.
- **태스크 큐(Task Queue):** 웹 API에서 처리가 끝난 콜백 함수가 대기하는 줄입니다.

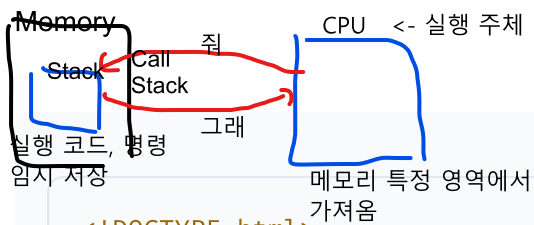
콜 스택이 할 일이 없을 때
이벤트 루프는 콜 스택이 비어 있을 때마다 태스크 큐에서 작업을 가져와 실행합니다. 한 명의 요리사(콜 스택)가 주문 대기열(태스크 큐)을 보며 손이 비는 대로 다음 주문을 처리하는 것과 같습니다.



JavaScript 이벤트 루프 동작 원리

setTimeout으로 비동기 체험하기 타임아웃 세팅해라.

다음 코드는 `setTimeout` 을 사용하여 이벤트 루프의 실행 순서를 직접 확인합니다.



js엔진은 single thread. -> 동기 처리만 가능

실행 환경 (브라우저, Node.js) multi thread 가능 -> 비동기 가능

js 엔진(v8)

하나의 호출 스택(Call Stack)
js는 동기만 됨..

메모리 영역
CPU가 호출하는 영역

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>이벤트 루프 실험</title>
</head>
<body>
  <script>
```

```
    console.log('1: 동기 코드 시작');
```

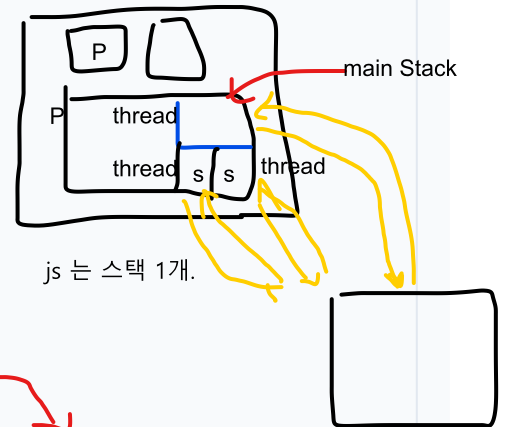
```
    setTimeout(() => {
      console.log('3: setTimeout 콜백 (2초 뒤)');
    }, 2000);
```

```
    setTimeout(() => {
      console.log('2: setTimeout 콜백 (0초 뒤)');
    }, 0);
```

```
    console.log('1: 동기 코드 끝');
```

```
</script>
</body>
</html>
```

프로세스



js 는 스택 1개.

CPU
C -> B -> A
순으로 실행

if 처리 안 끝났으면
task queue로 돌아감

```
// 콘솔 출력:
// 1: 동기 코드 시작
// 1: 동기 코드 끝
// 2: setTimeout 콜백 (0초 뒤)
// 3: setTimeout 콜백 (2초 뒤)
```

줄별 코드 설명

줄	코드	설명
1	<code>console.log('1: 동기 코드 시작')</code>	콜 스택에 즉시 쌓이고 실행됩니다
3	<code>setTimeout(() => { ... }, 2000)</code>	2초 타이머를 웹 API에 등록하고 즉시 다음 줄로 넘어갑니다. 콜 스택을 막지 않습니다
7	<code>setTimeout(() => { ... }, 0)</code>	0초 타이머도 웹 API를 거쳐 태스크 큐로 갑니다. 동기 코드가 모두 끝난 뒤에 실행됩니다
11	<code>console.log('1: 동기 코드 끝')</code>	동기 코드가 모두 실행된 후, 이벤트 루프가 태스크 큐의 콜백을 처리합니다

핵심: `setTimeout(fn, 0)` 은 "즉시 실행"이 아닙니다. 현재 동기 코드가 전부 끝난 뒤에야 실행됩니다. 이것이 이벤트 루프의 핵심 동작입니다.

마이크로태스크 큐(Microtask Queue): Promise의 `.then()` 콜백은 일반 태스크 큐보다 우선순위가 높은 마이크로태스크 큐에 들어갑니다. 따라서 `setTimeout` 콜백보다 먼저 실행됩니다. 이 점을 기억해 두면 나중에 실행 순서 관련 문제를 해결하는 데 도움이 됩니다.

절 요약

- JavaScript는 단일 스레드로 한 번에 하나의 작업만 처리합니다
- 이벤트 루프는 콜 스택이 빌 때 태스크 큐의 작업을 가져와 실행합니다
- `setTimeout(fn, 0)` 도 현재 동기 코드가 끝난 뒤에 실행됩니다
- Promise 콜백(`.then`)은 마이크로태스크 큐를 통해 `setTimeout` 보다 먼저 실행됩니다

비동기에서의

콜백과 프로미스

도입

비동기 작업이 끝난 뒤 "**그 다음에 할 일**"을 어떻게 지정할까요? 가장 기본적인 방법이 **콜백(Callback)** 함수입니다. 그런데 콜백을 여러 번 중첩하면 코드가 깊어지고 읽기 어려워집니다. 이 문제를 해결하기 위해 **프로미스(Promise)** 가 도입되었습니다.

핵심 개념 설명

콜백(Callback)과 콜백 지옥(Callback Hell)

콜백(Callback)은 "나중에 호출해 달라고 넘겨두는 함수"입니다. 비동기 작업이 끝나면 브라우저가 이 함수를 호출합니다.

지금은 데이터 없지만 미래의 어느 시점에 작업 성공/실패 시 결과 돌려주겠다! 약속

해결: Promise 객체

다음 코드는 콜백이 중첩될 때 발생하는 콜백 지옥을 보여줍니다.

비동기 처리 로직을 구현하기 위해
콜백 함수를 연속적으로 사용할 때
코드가 깊어지고 중첩되어 복잡해지는 현상

문제점
- 가독성 낮아짐
- 에러 처리 불가능
-> 유지보수 어렵

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>콜백 지옥 예시</title>
</head>
<body>
  <script>
    // 1단계: 사용자 정보를 가져온다 (0.5초 걸림)
    setTimeout(() => {
      const userId = 42;
      console.log('1단계: 사용자 ID 획득', userId);

      // 2단계: 해당 사용자의 주문 목록을 가져온다 (0.5초 걸림)
      setTimeout(() => {
        const orders = ['상품A', '상품B'];
        console.log('2단계: 주문 목록', orders);

        // 3단계: 첫 번째 주문의 배송 상태를 가져온다 (0.5초 걸림)
        setTimeout(() => {
          const status = '배송 중';
          console.log('3단계: 배송 상태', status);
          // 콜백이 깊어질수록 들여쓰기가 쌓이고 유지보수가 어려워집니다
        }, 500);
      }, 500);
    }, 500);
  </script>
</body>
</html>
```

위가 있어야 아래가 실행됨.
그래서 중첩

```
// 콘솔 출력 :
// 1단계: 사용자 ID 획득 42
// 2단계: 주문 목록 ['상품A', '상품B']
// 3단계: 배송 상태 배송 중
```

줄별 코드 설명

줄	코드	설명
1	첫 번째 <code>setTimeout</code>	비동기 1단계 작업을 시뮬레이션합니다
6	두 번째 <code>setTimeout</code> (중첩)	1단계 완료 후 2단계를 실행합니다. 들여쓰기가 한 단계 더 깊어집니다
12	세 번째 <code>setTimeout</code> (이중 중첩)	2단계 완료 후 3단계를 실행합니다. 이처럼 중첩이 계속되면 코드를 읽기 매우 어려워집니다

프로미스(Promise)의 세 가지 상태

프로미스(Promise)는 "미래에 완료될 작업"을 나타내는 객체입니다. 음식 주문 후 받는 진동 벨처럼, 지금은 결과가 없지만 완료되면 알려 주겠다는 약속과 같습니다.

프로미스는 세 가지 상태를 가집니다.

상태	의미	전환 조건
대기(pending)	아직 결과를 모름 완료x	초기 상태
이행(fulfilled)	작업이 성공적으로 완료됨 완료o->성공 결과	<code>resolve()</code> 호출 시
거부(rejected)	작업이 실패함 완료o -> 실패 결과	<code>reject()</code> 호출 시

Promise 상태 전이 다이어그램

then·catch·finally로 결과 다루기

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Promise 기초</title>
</head>
<body>
  <script>
    // Promise 생성: 2초 뒤 성공 또는 실패
    const fetchData = new Promise((resolve, reject) => {
      setTimeout(() => {
        const success = true; // false로 바꾸면 실패 케이스 확인 가능
        if (success) {
          resolve({ id: 1, name: '홍길동' });
        } else {
          reject(new Error('데이터 로드 실패'));
        }
      }, 2000);
    });

    fetchData
      .then((data) => {
        console.log('성공:', data.name); // fulfilled 상태에서 실행
        return data.id; // 다음 then으로 값 전달 (채이닝)
      })
      .then((id) => {
        console.log('ID 확인:', id); // 이전 then의 반환값을 받음
      })
      .catch((error) => {
        console.error('오류:', error.message); // rejected 상태에서 실행
      })
      .finally(() => {
        console.log('항상 실행됨'); // 성공/실패 무관하게 항상 실행
      });

    console.log('Promise 상태 확인 (pending):', fetchData);
  </script>
</body>
</html>

```

```
// 콘솔 출력 (success = true인 경우):
// Promise 상태 확인 (pending): Promise {<pending>}
// (2초 뒤)
// 성공: 홍길동
// ID 확인: 1
// 항상 실행됨
```

줄별 코드 설명

줄	코드	설명
2	<code>new Promise((resolve, reject) ...</code>	Promise 객체를 생성합니다. 인자로 받은 함수(executor)가 즉시 실행됩니다
4	<code>const success = true</code>	성공/실패 조건입니다. 실제 API 호출에서는 서버 응답 결과가 이 역할을 합니다
6	<code>resolve({ id: 1, name: '홍길동' ...</code>	fulfilled 상태로 전환하고, 전달한 값이 <code>.then()</code> 콜백의 인자가 됩니다
8	<code>reject(new Error(' ... '))</code>	rejected 상태로 전환하고, <code>.catch()</code> 콜백의 인자가 됩니다
13	<code>.then((data) => { ... })</code>	fulfilled 상태일 때만 실행됩니다. 반환값은 다음 <code>.then()</code> 으로 전달됩니다
18	<code>.catch((error) => { ... })</code>	rejected 상태이거나 <code>.then()</code> 안에서 오류가 발생했을 때 실행됩니다
21	<code>.finally(() => { ... })</code>	성공·실패에 관계없이 항상 실행됩니다. 로딩 스피너 숨기기 등에 활용합니다

팁: `.then()` 안에서 값을 `return` 하면 다음 `.then()` 이 그 값을 받아 체이닝이 이어집니다. 이를 **프로미스 체이닝(Promise Chaining)** 이라고 합니다.

주의: `Promise` 를 생성하면 executor 함수가 즉시 실행됩니다. 비동기 작업이 `.then()` 에서 시작된다고 오해하지 않도록 주의하십시오.

절 요약

- 콜백을 중첩하면 코드가 깊어지는 콜백 지옥이 발생합니다
- `Promise` 는 pending → fulfilled 또는 rejected 상태로 전이합니다
- `.then()` 은 성공, `.catch()` 는 실패, `.finally()` 는 항상 실행됩니다
- `.then()` 의 반환값으로 프로미스 체이닝이 가능합니다

async/await로 깔끔하게 쓰기

도입

프로미스 체이닝은 콜백 지옥보다 훨씬 낫지만, `.then().then().then()` 이 길어지면 여전히 읽기 불편합니다. **async/await** 문법을 사용하면 **비동기 코드**를 마치 동기 코드처럼 위에서 아래로 읽히도록 작성할 수 있습니다.

핵심 개념 설명

async 함수와 await

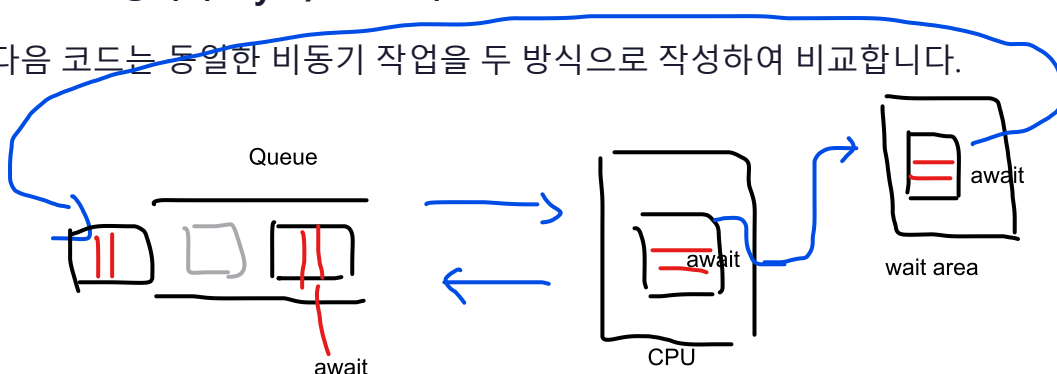
`async` 키워드를 함수 앞에 붙이면 그 함수는 **항상 프로미스를 반환**하는 **비동기 함수**가 됩니다. `await` 는 `async` 함수 안에서만 사용할 수 있으며, **프로미스가 완료될 때까지 해당 함수의 실행을 일시 중단**합니다.

줄을 서서 차례를 기다렸다가 결과를 받고 다음 일을 진행하는 것처럼, 코드를 순서대로 읽게 해 줍니다.

중요한 점은 `await` 가 **전체 프로그램을 멈추는 것이 아닙니다**. 해당 `async` 함수의 실행만 일시 중단되고, **다른 코드는 계속 실행**됩니다.

Promise 방식과 async/await 비교

다음 코드는 동일한 비동기 작업을 두 방식으로 작성하여 비교합니다.



```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>async/await vs Promise</title>
</head>
<body>
  <script>
    // 비동기 작업을 시뮬레이션하는 헬퍼 함수
    const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

    // --- 방식 1: Promise 체이닝 ---
    function loadUserWithPromise(userId) {
      return delay(500)
        .then(() => ({ id: userId, name: '홍길동' })))
        .then((user) => {
          console.log('[Promise] 사용자:', user.name);
          return delay(500).then(() => ['주문A', '주문B']);
        })
        .then((orders) => {
          console.log('[Promise] 주문 목록:', orders);
        })
        .catch((error) => {
          console.error('[Promise] 오류:', error.message);
        });
    }
  
```

1. Async

async function
Promise(return 처리 결과)
↑
async function의 결과는
Promise 오브젝트가 됨.
Promise.resolve(값)과 같음.

성공한 결과값만
빼내기 위해 사용

2. await

- 비동기 작업 끝날 때까지
코드 실행 중지

```

    // --- 방식 2: async/await ---
    async function loadUserWithAsync(userId) {      async는 catch 따로 없어서 try-catch해야.
      try {
        await delay(500);      server 다녀오는 시간
        const user = { id: userId, name: '홍길동' };
        console.log('[async] 사용자:', user.name);

        await delay(500);      server 다녀오는 시간
        const orders = ['주문A', '주문B'];
        console.log('[async] 주문 목록:', orders);
      } catch (error) {
        console.error('[async] 오류:', error.message);      Promise.reject(값)
      }
    }
  
```

loadUserWithPromise(1); -> 0.5초 휴식 -> 홍길동 -> 0.5초 휴식 -> order -> 종료
loadUserWithAsync(2); -> 실행 -> 0.5초 휴식 -> 홍길동 -> 0.5초 휴식 -> 오더 -> 종료

async: 함수 밖 비동기

```

</script>
</body>
</html>

```

정상실행 => 결과값 => 변수
비정상실행(error) => catch

async function() {
 변수 = await function()
 코드
 코드

중지 후 결과 변수에 대입'
await 없으면 pending으로 처리되고
아래 코드 실행되어 오류 유발

Promise.all(func1, func2, func3] <- 셋 다 비동기 처리(조건: 세 함수 모두 async)
Promise.any() : 가장 먼저 성공한 Promise의 결과값만 받는 기능(1개만 받음)

동시 실행

```
// 콘솔 출력:  
// [Promise] 사용자: 홍길동  
// [async] 사용자: 홍길동  
// (두 방식이 거의 동시에 출력됨. 각 내부는 0.5초 간격)  
// [Promise] 주문 목록: ['주문A', '주문B']  
// [async] 주문 목록: ['주문A', '주문B']
```

줄별 코드 설명

줄	코드	설명
2	<code>const delay = (ms) => new Prom...</code>	지정한 밀리초 뒤 resolve되는 헬퍼 함수입니다. 실제 API 호출 대신 지연을 시뮬레이션합니다
5~19	Promise 체이닝 방식	<code>.then()</code> 이 중첩되어 가로로 긴 코드가 됩니다
22	<code>async function loadUserWithAsy...</code>	<code>async</code> 키워드로 비동기 함수를 선언합니다. 이 함수는 항상 Promise를 반환합니다
24	<code>try {</code>	<code>await</code> 코드에서 발생하는 오류를 잡기 위해 <code>try/catch</code> 로 감쌉니다
25	<code>await delay(500)</code>	0.5초를 기다립니다. 이 줄에서 함수 실행이 일시 중단되고, 완료 후 다음 줄로 넘어갑니다
30	<code>} catch (error) {</code>	비동기 작업 중 오류가 발생하면 여기서 처리합니다

try/catch로 에러 처리하기

`await` 가 reject된 프로미스를 받으면 에러를 던집니다. 이를 `try/catch` 로 잡습니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>async/await 에러 처리</title>
</head>
<body>
  <script>
    async function riskyOperation() {
      try {
        // 50% 확률로 실패하는 작업 시뮬레이션
        const result = await new Promise((resolve, reject) => {
          setTimeout(() => {
            if (Math.random() > 0.5) {
              resolve('작업 성공!');
            } else {
              reject(new Error('작업 실패: 서버 오류'));
            }
          }, 1000);
        });

        console.log('결과:', result);
      } catch (error) {
        // rejected 상태의 에러가 여기서 잡힙니다
        console.error('오류 처리:', error.message);
      } finally {
        console.log('작업 완료 (성공·실패 무관)');
      }
    }

    riskyOperation();
  </script>
</body>
</html>

```

```

// 콘솔 출력 (성공한 경우):
// 결과: 작업 성공!
// 작업 완료 (성공·실패 무관)

// 콘솔 출력 (실패한 경우):
// 오류 처리: 작업 실패: 서버 오류
// 작업 완료 (성공·실패 무관)

```

줄별 코드 설명

줄	코드	설명
1	<code>async function riskyOperation()</code>	async 함수를 선언합니다
2	<code>try {</code>	<code>await</code> 블록 내에서 오류가 발생할 수 있으므로 <code>try</code> 로 시작합니다
3	<code>const result = await new Promi...</code>	프로미스가 완료될 때까지 기다립니다. resolve되면 값이, reject되면 <code>catch</code> 로 제어가 넘어갑니다
11	<code>resolve('작업 성공!')</code>	성공 시 <code>result</code> 변수에 이 값이 담깁니다
13	<code>reject(new Error(' ... '))</code>	실패 시 <code>catch</code> 블록의 <code>error</code> 로 이 Error 객체가 전달됩니다
19	<code>} catch (error) {</code>	<code>await</code> 가 reject된 Promise를 받으면 이 블록이 실행됩니다

베스트 프랙티스: `await` 를 사용하는 코드는 항상 `try/catch` 로 감싸십시오. 에러 처리를 빠뜨리면 아무런 알림 없이 작업이 실패할 수 있습니다.

비교 테이블

구분	콜백	Promise (.then)	async/await
가독성	중첩 시 나쁨	보통	좋음 (동기 코드처럼 읽힘)
에러 처리	각 단계마다	<code>.catch()</code>	<code>try/catch</code>
디버깅	어려움	보통	쉬움
지원 환경	모든 환경	ES2015+	ES2017+ (모던 브라우저)

절 요약

- `async` 함수는 항상 프로미스를 반환합니다
- `await` 는 프로미스가 완료될 때까지 해당 함수의 실행을 일시 중단합니다

- `try/catch` 로 비동기 오류를 동기 코드처럼 처리합니다
- `finally` 는 성공·실패에 관계없이 항상 실행됩니다

Fetch API로 데이터 가져오기

도입

이론을 익혔으니 이제 실제 인터넷에서 데이터를 가져와 봅시다. **Fetch API** 는 브라우저에 내장된 표준 네트워크 요청 인터페이스입니다. 음식점에 전화로 주문(요청)을 넣고 배달(응답)을 받는 것과 같습니다. `fetch()` 는 프로미스를 반환하기 때문에 `async/await` 와 자연스럽게 결합됩니다.

핵심 개념 설명

fetch로 요청 보내기와 응답 상태 확인하기

`fetch(url)` 을 호출하면 `Response` 객체를 담은 프로미스를 반환합니다. 이 `Response` 객체에는 상태 코드(`status`)와 성공 여부(`ok`)가 담겨 있습니다.

catch로 잡기 어렵..

중요한 주의사항: `fetch` 는 404, 500 같은 HTTP 오류 상태 코드를 받아도 reject되지 않습니다. 네트워크 연결 자체가 실패할 때만 reject됩니다. 따라서 `response.ok` 를 반드시 확인해야 합니다.

Fetch API HTTP 요청-응답 흐름

```
fetch(resource, options?)
```

항목	내용
설명	네트워크 요청을 시작하고 응답을 Promise로 반환하는 브라우저 내장 함수입니다
매개변수	<code>resource</code> (string Request): 요청할 URL 또는 Request 객체
	<code>options</code> (object, 선택): <code>method</code> , <code>headers</code> , <code>body</code> 등 요청 옵션
반환값	<code>Promise<Response></code> - 응답 객체를 담은 프로미스
예외	네트워크 오류 또는 CORS 정책 위반 시 <code>reject</code>
주의	HTTP 4xx/5xx 응답은 <code>reject</code> 되지 않으므로 <code>response.ok</code> 를 직접 확인해야 합니다

다음 코드는 JSONPlaceholder(무료 공개 테스트 API)에서 사용자 목록을 가져옵니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Fetch API 기초</title>
</head>
<body>
  <script>
    async function getUsers() {
      try {
        // 1단계: 요청 보내기
        const response = await fetch('https://jsonplaceholder.typicode.com/users');
        // response = [{}, {}, {}] 이렇게 들어와도 외부에서 들어올 때는 문자열로 들어옴.

        // 2단계: 응답 상태 확인 (반드시 필요!)
        if (!response.ok) {
          // 404, 500 등
          throw new Error(`HTTP 오류: 상태 코드 ${response.status}`);
        }

        // 3단계: JSON으로 파싱 문자열 -> JSON
        const users = await response.json();

        // 4단계: 데이터 사용
        console.log(`총 ${users.length}명의 사용자를 불러왔습니다`);
        users.slice(0, 3).forEach((user) => {
          console.log(`- ${user.name} (${user.email})`);
        });
      } catch (error) {
        console.error('오류 발생:', error.message);
      }
    }

    getUsers();
  </script>
</body>
</html>

```

```

// 콘솔 출력:
// 총 10명의 사용자를 불러왔습니다
// - Leanne Graham (Sincere@april.biz)
// - Ervin Howell (Shanna@melissa.tv)
// - Clementine Bauch (Nathan@yesenia.net)

```

줄별 코드 설명

줄	코드	설명
2	<code>async function getUsers()</code>	비동기 함수를 선언합니다. <code>await</code> 를 사용하려면 반드시 <code>async</code> 함수여야 합니다
3	<code>try {</code>	네트워크 오류 및 <code>throw</code> 된 오류를 잡기 위해 <code>try/catch</code> 로 감쌉니다
5	<code>const response = await fetch(...</code>	URL로 GET 요청을 보내고, 응답 헤더가 도착할 때까지 기다립니다
8	<code>if (!response.ok)</code>	<code>response.ok</code> 는 상태 코드가 200~299 범위일 때 <code>true</code> 입니다. 이 확인을 생략하면 404 나 500 응답도 정상으로 처리됩니다
9	<code>throw new Error(...)</code>	수동으로 오류를 던져 <code>catch</code> 블록이 처리하게 합니다
13	<code>const users = await response.j...</code>	응답 본문을 JSON으로 파싱합니다. 이것도 비동기 작업이므로 <code>await</code> 가 필요합니다
17	<code>users.slice(0, 3).forEach(...)</code>	처음 3명의 정보만 출력합니다

에러와 네트워크 실패 처리하기

실제 서비스에서는 다양한 오류 상황을 처리해야 합니다. 다음 코드는 여러 오류 상황을 체계적으로 다루는 방법을 보여줍니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Fetch 에러 처리</title>
</head>
<body>
  <script>
    async function fetchWithErrorHandling(url) {
      try {
        const response = await fetch(url);

        if (!response.ok) {
          // HTTP 오류: 404(찾을 수 없음), 500(서버 오류) 등
          const message = `요청 실패 (상태 코드: ${response.status})`;
          throw new Error(message);
        }

        return await response.json();
      } catch (error) {
        if (error instanceof TypeError) {
          // TypeError: 네트워크 연결 실패, CORS 오류, URL이 잘못된 경우
          console.error('네트워크 오류:', error.message);
        } else {
          // HTTP 오류 또는 JSON 파싱 오류
          console.error('요청 오류:', error.message);
        }
        return null; // 오류 시 null 반환
      }
    }

    // 정상 요청
    fetchWithErrorHandling('https://jsonplaceholder.typicode.com/posts/1')
      .then((data) => data && console.log('게시글 제목:', data.title));

    // 존재하지 않는 리소스 (404 응답)
    fetchWithErrorHandling('https://jsonplaceholder.typicode.com/posts/99999')
      .then((data) => data && console.log('데이터:', data));
  </script>
</body>
</html>

```

```
// 콘솔 출력:  
// 게시글 제목: sunt aut facere repellat provident occaecati excepturi optio re  
// 요청 오류: 요청 실패 (상태 코드: 404 Not Found)
```

줄별 코드 설명

줄	코드	설명
1	<code>async function fetchWithErrorH...</code>	URL을 인자로 받아 재사용 가능한 fetch 래퍼 함수입니다
4	<code>if (!response.ok)</code>	HTTP 오류 상태 코드를 직접 확인합니다
5	<code>`상태 코드: \${response.status} ...</code>	상태 코드(숫자)와 상태 텍스트("Not Found")를 함께 출력합니다
10	<code>if (error instanceof TypeError)</code>	네트워크 연결 실패는 <code>TypeError</code> 로 나타납니다. HTTP 오류와 구분하여 처리합니다
15	<code>return null</code>	오류 시 <code>null</code> 을 반환하여 호출하는 쪽에서 결과를 확인하게 합니다

베스트 프랙티스: `fetch` 호출은 항상 이 두 가지를 갖추어야 합니다. 첫째, `try/catch`로 네트워크 오류를 처리하십시오. 둘째, `if (!response.ok)`로 HTTP 오류를 처리하십시오. 이 두 가지를 모두 갖추어야 견고한 코드가 됩니다.

절 요약

- `fetch(url)`은 `Response` 객체를 담은 프로미스를 반환합니다
- `response.ok`로 HTTP 성공 여부를 반드시 확인해야 합니다
- `response.json()`도 비동기 작업이므로 `await`가 필요합니다
- `TypeError`는 네트워크 실패, 그 외는 HTTP 오류나 파싱 오류를 의미합니다

JSON 처리와 간단한 앱 만들기

도입

서버에서 받아온 데이터는 거의 항상 **JSON(JavaScript Object Notation)** 형식입니다. **JSON**을 **JavaScript 객체로 변환**하고, 반대로 **JavaScript 객체를 JSON으로 바꾸는 방법**을 익힙니다. 그런 다음 지금까지 배운 것을 모두 합쳐 실제로 작동하는 미니 앱을 만들어 봅니다.

핵심 개념 설명

JSON 구조와 JavaScript 객체의 관계

JSON(JavaScript Object Notation)은 ^{객체} ^{표기법} **키-값 쌍**으로 데이터를 표현하는 경량 **텍스트 형식**입니다. 서로 다른 나라 사람도 알아보는 표준 양식의 서류처럼, 시스템 간에 통용되는 공통 데이터 포맷입니다.

JavaScript 객체와 매우 비슷하지만 차이점이 있습니다.

구분	JSON	JavaScript 객체
형태	텍스트 문자열	메모리의 객체
키 표기	반드시 큰따옴표	따옴표 생략 가능
값 타입	문자열·숫자·불리언·null·배열·객체만 가능	함수·undefined 등도 가능
주석	불가	가능

JSON.parse와 JSON.stringify

JSON.parse(text)

항목	내용
설명	JSON 문자열을 JavaScript 값·객체로 변환합니다
매개변수	<code>text</code> (string): 파싱할 JSON 문자열
반환값	JavaScript 값 또는 객체
예외	유효하지 않은 JSON이면 <code>SyntaxError</code> 를 던집니다
사용 예시	<code>JSON.parse('{ "name": "홍길동" }')</code> → <code>{ name: '홍길동' }</code>

JSON.stringify(value, replacer?, space?)

항목	내용
설명	JavaScript 값·객체를 JSON 문자열로 변환합니다
매개변수	<code>value</code> : 변환할 JavaScript 값
	<code>replacer</code> (선택): 포함할 속성을 지정하는 함수 또는 배열
	<code>space</code> (number string, 선택): 들여쓰기 공백 수
반환값	JSON 형식의 문자열
예외	순환 참조가 있으면 <code>TypeError</code> 를 던집니다
사용 예시	<code>JSON.stringify({ name: '홍길동' }, null, 2)</code>

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>JSON 처리</title>
</head>
<body>
  <script>
    // JSON 문자열 (서버에서 받은 텍스트라고 가정)
    const jsonString = '{"id":1,"name":"홍길동","skills":["HTML","CSS","JS"]}';

    // JSON.parse: 문자열 → JavaScript 객체
    const user = JSON.parse(jsonString);
    console.log(typeof user);           // "object"
    console.log(user.name);             // "홍길동"
    console.log(user.skills[0]);        // "HTML"

    // 객체 수정
    user.skills.push('Python');

    // JSON.stringify: JavaScript 객체 → JSON 문자열
    const updatedJson = JSON.stringify(user, null, 2); // 2칸 들여쓰기
    console.log(typeof updatedJson);    // "string"
    console.log(updatedJson);
  </script>
</body>
</html>

```

```

// 콘솔 출력:
// object
// 홍길동
// HTML
// string
// {
//   "id": 1,
//   "name": "홍길동",
//   "skills": [
//     "HTML",
//     "CSS",
//     "JS",
//     "Python"
//   ]
// }

```

줄별 코드 설명

줄	코드	설명
2	<code>const jsonString = ' ... '</code>	서버에서 받아온 JSON 텍스트를 시뮬레이션합니다. 실제로는 <code>response.json()</code> 이 파싱을 자동으로 처리합니다
5	<code>JSON.parse(jsonString)</code>	텍스트를 JavaScript 객체로 변환합니다. 이 시점부터 점 표기법으로 속성에 접근할 수 있습니다
12	<code>user.skills.push('Python')</code>	변환된 객체는 일반 JavaScript 객체이므로 자유롭게 수정할 수 있습니다
15	<code>JSON.stringify(user, null, 2)</code>	객체를 JSON 문자열로 변환합니다. 세 번째 인자 <code>2</code> 는 들여쓰기 칸 수로, 읽기 쉬운 형태로 출력합니다

가져온 데이터를 DOM에 렌더링하기

이제 지금까지 배운 모든 것을 종합하여 공개 API에서 사용자 목록을 가져와 화면에 표시하는 앱을 만들겠습니다. 로딩 중 상태와 오류 상태도 함께 처리합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>사용자 목록 앱</title>
  <style>
    body {
      font-family: sans-serif;
      max-width: 600px;
      margin: 2rem auto;
      padding: 0 1rem;
    }

    #status {
      padding: 0.75rem 1rem;
      border-radius: 6px;
      margin-bottom: 1rem;
    }

    #status.loading {
      background: #e3f2fd;
      color: #1565c0;
    }

    #status.error {
      background: #ffebee;
      color: #c62828;
    }

    #status.hidden {
      display: none;
    }

    .user-card {
      border: 1px solid #e0e0e0;
      border-radius: 8px;
      padding: 1rem;
      margin-bottom: 0.75rem;
    }

    .user-card h3 {
      margin: 0 0 0.25rem 0;
      font-size: 1rem;
    }

    .user-card p {
      margin: 0;
      color: #666;
      font-size: 0.875rem;
    }
  </style>
</head>
<body>
  <div id="status" class="loading">
    <div class="user-card">
      <h3>사용자 목록</h3>
      <p>사용자 목록 앱</p>
    </div>
  </div>
</body>
</html>
```



```

    }
  </style>
</head>
<body>
  <h1>사용자 목록</h1>

  <!-- 로딩/에러 상태 표시 영역 -->
  <div id="status" class="loading">데이터를 불러오는 중...</div>

  <!-- 사용자 목록 표시 영역 -->
  <div id="user-list"></div>

<script>
  const statusEl = document.getElementById('status');
  const userListEl = document.getElementById('user-list');

  // 로딩 상태 표시 함수
  function showLoading() {
    statusEl.textContent = '데이터를 불러오는 중...';
    statusEl.className = 'loading';
  }

  // 오류 상태 표시 함수
  function showError(message) {
    statusEl.textContent = `오류: ${message}`;
    statusEl.className = 'error';
  }

  // 상태 숨기기 함수
  function hideStatus() {
    statusEl.className = 'hidden';
  }

  // 사용자 카드 DOM 생성 함수 (innerHTML 대신 createElement 사용 - XSS 방지)
  function createUserCard(user) {
    const card = document.createElement('div');
    card.className = 'user-card';

    const name = document.createElement('h3');
    name.textContent = user.name; // textContent로 안전하게 삽입

    const email = document.createElement('p');
    email.textContent = user.email;

    const company = document.createElement('p');
    company.textContent = `회사: ${user.company.name}`;

    card.append(name, email, company);
    return card;
  }

  // 메인 비동기 함수

```

```

async function loadUsers() {
  showLoading();

  try {
    const response = await fetch(
      'https://jsonplaceholder.typicode.com/users'
    );

    if (!response.ok) {
      throw new Error(`서버 오류 (${response.status})`);
    }

    const users = await response.json();

    // DOM 렌더링
    const fragment = document.createDocumentFragment(); // 성능 최적화
    users.forEach((user) => {
      fragment.appendChild(createUserCard(user));
    });
    userListEl.appendChild(fragment);

    hideStatus();
  } catch (error) {
    showError(error.message);
  }
}

loadUsers();
</script>
</body>
</html>

```

브라우저에서 열면 "데이터를 불러오는 중..." 메시지가 표시되다가, API 응답이 도착하면 사용자 카드 목록이 나타납니다.

줄별 코드 설명

줄	코드	설명
<code>showLoading()</code>	상태 표시 함수	로딩 시작 시 호출합니다. 클래스를 바꿔 스타일을 전환합니다
<code>showError(message)</code>	오류 상태 함수	오류 발생 시 메시지를 화면에 표시합니다
<code>createUserCard(user)</code>	DOM 생성 함수	<code>textContent</code> 를 사용하여 사용자 입력값이 HTML로 해석되지 않게 합니다 (XSS 방지)
<code>createDocumentFragment()</code>	성능 최적화	여러 요소를 DOM에 추가할 때 Fragment를 사용하면 리플로우를 한 번만 발생시킵니다
<code>await fetch(...)</code>	네트워크 요청	응답 헤더 도착까지 기다립니다
<code>if (!response.ok)</code>	상태 코드 확인	HTTP 오류를 수동으로 throw하여 catch 블록에서 처리합니다
<code>await response.json()</code>	JSON 파싱	응답 본문을 JavaScript 배열로 변환합니다
<code>hideStatus()</code>	상태 숨기기	성공적으로 렌더링이 완료되면 로딩 메시지를 숨깁니다

보안 주의: 서버에서 받아온 데이터를 DOM에 삽입할 때는 `innerHTML` 대신 `textContent` 를 사용하십시오. `innerHTML` 을 사용하면 악의적인 스크립트가 포함된 데이터가 실행되는 XSS(Cross-Site Scripting) 취약점이 생길 수 있습니다.

팁: `DocumentFragment` 는 실제 DOM에 연결되지 않은 임시 컨테이너입니다. 여기에 요소를 추가하고 마지막에 한 번만 DOM에 삽입하면, 브라우저가 레이아웃을 매번 다시 계산하는 비용을 줄일 수 있습니다.

절 요약

- JSON은 텍스트 문자열이며, 키는 반드시 큰따옴표로 감싸야 합니다
- `JSON.parse()` 는 JSON 문자열을 JavaScript 객체로 변환합니다
- `JSON.stringify()` 는 JavaScript 객체를 JSON 문자열로 변환합니다
- 데이터를 DOM에 삽입할 때는 `textContent` 를 사용하여 XSS를 방지합니다
- 로딩·오류·성공 세 가지 상태를 항상 사용자에게 보여 주어야 합니다

FAQ

Q1: `fetch` 가 404 응답을 받아도 `catch` 가 실행되지 않는 이유는 무엇인가요?

A1: `fetch` 는 네트워크 연결 자체가 실패할 때만 reject됩니다. 서버가 존재하고 응답을 돌려줬다면, 그 응답이 404나 500이어도 `fetch` 는 정상적으로 완료(resolve)됩니다. 따라서 `response.ok` 를 직접 확인하고 필요하면 `throw new Error()` 로 오류를 만들어야 합니다. 이것이 `fetch` 를 사용할 때 가장 흔한 실수입니다.

Q2: `await` 를 `async` 함수 바깥에서 사용하면 어떻게 되나요?

A2: 문법 오류(SyntaxError)가 발생합니다. `await` 는 반드시 `async` 로 선언된 함수 안에서만 사용할 수 있습니다. 최상위 레벨(top-level)에서 `await` 를 사용하려면 ES2022의 Top-Level Await를 지원하는 모듈 환경이 필요합니다. 일반 브라우저 스크립트에서는 항상 `async` 함수로 감싸 사용하십시오.

Q3: `Promise.all()` 은 무엇이고 언제 사용하나요?

A3: 여러 프로미스를 동시에 실행하고 모두 완료될 때까지 기다리는 메서드입니다. 예를 들어 사용자 정보와 상품 목록을 동시에 가져올 때 사용합니다. `await Promise.all([fetchUsers(), fetchProducts()])` 처럼 쓰면 두 요청이 순차가 아닌 병렬로 실행되어 전체 대기 시간이 줄어듭니다. 단, 하나라도 실패하면 전체가 reject됩니다.

Q4: 콜백, 프로미스, `async/await` 중 어느 것을 써야 하나요?

A4: 새로 작성하는 코드에는 `async/await` 를 사용하는 것을 권장합니다. 가독성이 가장 좋고 디버깅도 쉽습니다. `async/await` 는 프로미스를 기반으로 하므로, 프로미스를 반환하는 API 라면 모두 `await` 로 처리할 수 있습니다. 콜백은 `setTimeout` , `addEventListener` 같이 API 자체가 콜백 기반일 때 어쩔 수 없이 사용합니다.

Q5: 개발자 도구의 Network 탭에서 `fetch` 요청을 어떻게 확인하나요?

A5: Chrome/Edge 개발자 도구에서 F12를 누르고 Network 탭을 선택합니다. 페이지를 새로고침하거나 `fetch`를 실행하면 요청 목록이 표시됩니다. 요청 항목을 클릭하면 Headers(요청/응답 헤더), Preview(파싱된 JSON), Response(원본 텍스트), Timing(소요 시간) 등을 확인할 수 있습니다. 네트워크 디버깅 시 가장 먼저 확인해야 할 도구입니다.

장 요약

이번 장에서는 JavaScript의 비동기 처리 체계 전반을 학습했습니다. 이벤트 루프가 단일 스레드로 비동기를 처리하는 원리를 이해하고, 콜백에서 프로미스를 거쳐 `async/await` 로 발전해 온 비동기 코드 작성 방식을 익혔습니다. Fetch API를 사용하여 실제 공개 API를 호출하고, JSON을 파싱하여 DOM에 렌더링하는 완전한 데이터 흐름을 구현했습니다. 나아가 `response.ok` 확인, `try/catch` 에러 처리, `textContent` 사용을 통한 XSS 방지 등 실무에서 반드시 갖추어야 할 패턴도 함께 다루었습니다.

핵심 키워드

이벤트 루프, 콜 스택, 태스크 큐, Promise, async/await, Fetch API, response.ok, JSON.parse, JSON.stringify, try/catch

다음 장 미리보기

이번 장이 이 교재의 마지막 챕터입니다. 지금까지 HTML로 구조를, CSS로 표현을, JavaScript로 동작과 API 연동까지 다루며 프론트엔드 개발의 전체 흐름을 완성했습니다. 지금 배운 기초를 바탕으로 React, Vue 같은 현대적인 프레임워크를 학습하거나, 더 복잡한 웹 애플리케이션을 직접 만들어 보기를 권합니다.

✂ 실습 프로젝트 (Hands-on Project)

API 연동 미니 앱 프로젝트

이 장에서 배운 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: JSONPlaceholder 게시판 탐색기

시나리오

스타트업 팀에서 "공지사항 게시판" 프로토타입을 만들어 달라는 요청을 받았습니다. JSONPlaceholder 공개 API에서 게시글 목록을 가져와 화면에 표시하고, 특정 게시글을 클릭하면 상세 내용을 보여주는 간단한 탐색기를 완성합니다. 로딩 중과 오류 발생 시 사용자가 상황을 알 수 있도록 상태 메시지도 표시합니다.

학습 목표

- `async/await` + `try/catch`로 Fetch API 호출 구조 구현하기
- `response.ok` 확인으로 HTTP 오류를 안전하게 처리하기
- 로딩·오류·성공 세 가지 UI 상태를 전환하기
- `textContent`로 API 데이터를 DOM에 안전하게 렌더링하기

진행 방법

1. `project_starter.html` 을 열고 TODO 주석을 찾습니다
2. 각 TODO를 이 장에서 배운 `async/await`, Fetch, JSON 개념으로 완성합니다
3. 브라우저에서 열어 게시글 목록이 표시되고 클릭 시 상세 내용이 나오는지 확인합니다

실행 방법

`project_starter.html` 파일을 브라우저에서 열기
(인터넷 연결 필요: JSONPlaceholder API 사용)

사용 API

- 게시글 목록: `GET https://jsonplaceholder.typicode.com/posts?_limit=10`
- 게시글 상세: `GET https://jsonplaceholder.typicode.com/posts/{id}`
- 응답 구조: `{ id, title, body, userId }`

중급: Open-Meteo 날씨 앱

시나리오

사용자가 도시 이름을 입력하면 현재 날씨를 보여주는 미니 앱을 완성합니다. Open-Meteo API는 API 키 없이 사용할 수 있는 공개 날씨 서비스입니다. 도시 이름 → 위도/경도 변환(Geocoding API) → 날씨 데이터 조회 두 단계의 연속 API 호출을 `async/await`로 처리하며, 로딩·오류·성공 상태를 모두 구현합니다.

학습 목표

- 두 단계 연속 API 호출을 `async/await`로 순서대로 처리하기
- 여러 fetch 요청의 `response.ok`를 각각 확인하기
- 날씨 코드(WMO code)를 읽기 좋은 텍스트로 변환하는 데이터 매핑 구현하기

- 복잡한 JSON 구조에서 필요한 데이터 추출하기

진행 방법

1. `project_advanced_starter.html` 을 열고 TODO 주석을 찾습니다
2. 초급에서 연습한 패턴을 바탕으로 더 복잡한 두 단계 API 호출을 완성합니다
3. 도시 이름을 입력하고 버튼을 눌러 날씨 정보가 표시되는지 확인합니다

실행 방법

`project_advanced_starter.html` 파일을 브라우저에서 열기
(인터넷 연결 필요: Open-Meteo + Geocoding API 사용)

사용 API (모두 무료, API 키 불필요)

- **Geocoding:** `GET https://geocoding-api.open-meteo.com/v1/search?name={도시명}&count=1&language=ko`
- 응답 구조: `{ results: [{ name, latitude, longitude, country }] }`
- **날씨:** `GET https://api.open-meteo.com/v1/forecast?latitude={위도}&longitude={경도}¤t_weather=true`
- 응답 구조: `{ current_weather: { temperature, windspeed, weathercode } }`

날씨 코드(WMO) 참고

코드	날씨
0	맑음
1, 2, 3	대체로 맑음 ~ 흐림
45, 48	안개
51~67	이슬비 ~ 비
71~77	눈
80~82	소나기
95~99	천둥번개

확장 아이디어

- 여러 도시의 날씨를 `Promise.all`로 동시에 가져와 비교하기
- 검색 기록을 `localStorage`에 저장하여 최근 검색 도시 표시하기

- 날씨 코드에 따라 다른 배경 색상 또는 날씨 아이콘 표시하기

▶  project_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_complete.html (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)